simulator

node

utils

distributions

api

physics

fault_models

robot_model

state_estimators

sensors

fault_models

sensor_filters

environment

navigators controllers

networking

simba_com

scenario

gui

Simulator

from_config(SimulatorConfig, plugin_api: Option<PluginAPI>) -> Simulator
load_config(SimulatorConfig, plugin_api: Option<PluginAPI>)
reset(plugin_api: Option<PluginAPI>)
set_max_time(f32)
run()
load_results(filename: Option<String>) -> f32
deserialize_results_from_file(filename: Path) -> Results
compute_results()

get_records(sorted: bool = false) -> [Record]
get_async_api() -> SimulatorAsyncApi
get_broker() -> Broker
get_context() -> Context
get_environment() -> Environment
get_nodes() -> [Node]

SimulatorAsyncApi

current_time: f32
records: mpsc::Receiver<Record>

AsyncSimulator

from_config(SimulatorConfig, plugin_api: Option<PluginAPI>) -> AsyncSimulator
run(plugin_api: Option<PluginAPI>, max_time: Option<f32>, reset: bool)
get_records(sorted: bool = false) -> [Record]
compute_results()
stop()
get_simulator() -> Simulator

Node

next_time_step(min_time_excluded: f32) -> f32
name() -> String
state() -> NodeState
send_records() -> bool
other_node_names() -> [String]
node_type() -> NodeType
network() -> Network
physics() -> Physics
sensor_manager() -> SensorManager
state_estimator() -> StateEstimator
state_estimator_bench() -> [BenchStateEstimator]
navigator() -> Navigator
controller() -> Controller
environment() -> Environment
get_other_node_physics(node_name: String) -> Option<Physics>
get_all_node_physics() -> dict(String, Physics)
meta_data() -> NodeMetaData
meta_data_list() -> dict(String, NodeMetaData)
pre_kill()

NodeType

Robot
Sensor

Object
ComputationUnit

NodeFactory

make_robot(RobotConfig) -> Node
make_sensor(SensorConfig) -> Node
make_object(ObjectConfig) -> Node
make_computation_unit(ComputationUnitConfig) -> Node

NodeMetaData

name: String
node_type: NodeType
model_name: String
labels: [String]
state: NodeState

position: Option<(f32, f32)>

NodeState

Created
Running
Zombie
Terminated

DeterministRandomVariableFactory

DeterministRandomVariable

Frame

new(x: f32, y: f32, theta: f32) -> Frame
transform() -> Isometry2
attach_to_state(state: State) -> AttachedFrame

AttachedFrame

state() -> State
transform() -> Isometry2
transform_point_frame_to_global(point: (f32, f32)) -> (f32, f32)
transform_point_global_to_frame(point: (f32, f32)) -> (f32, f32)

Bernouilli Poisson Uniform Fixed Exponential Normal

AsyncApiRunner

new_with_simulator(Simulator) -> AsyncApiRunner
get_api() -> AsyncApi
get_simulator() -> Simulator
stop()
run(plugin_api: Option<PluginAPI>)

AsyncApi

simulator_api() -> SimulatorAsyncApi
load_config(SimulatorConfig, force_send_results: bool) -> SimulatorConfig
load_results(filename: Option<String>) -> f32
run(max_time: Option<f32>, reset: bool)
get_records(sorted: bool) -> [Record]
compute_results()

PluginAsyncAPI

get_client() -> PluginAsyncAPIClient

AsyncApiServer

The Async api of the simulator
use mpsc channels to communicate
between clients and simulator

PluginAsyncAPIClient

get_state_estimator(PluginAsyncAPIRequest) -> StateEstimator
get_navigator(PluginAsyncAPIRequest) -> Navigator
get_controller(PluginAsyncAPIRequest) -> Controller
get_physics(PluginAsyncAPIRequest) -> Physics

PluginAsyncAPIRequest

config: serde_json::Value
global_config: SimulatorConfig
va_factory: DeterministRandomVariableFactory
network: Network
initial_time: f32

Physics

post_init(Node)
apply_command(Command, time: f32)
update_state(time: f32)
state(time: f32) -> State
next_time_step() -> Option<f32>

Allow to be mobile
and to be perceived

PerfectPhysics ExternalPhysics PythonPhysics

PhysicsFaultModel

post_init(Node)
add_faults(time: f32, state: mut State)

AdditiveRobotCenteredPhysicsFault PythonPhysicsFaultModel

RobotModel

update_state(previous_state: mut State, command: Command, dt: f32)
default_command() -> Command

HolonomicCommand

longitudinal_velocity: f32
lateral_velocity: f32
angular_velocity: f32

Holonomic

UnicycleCommand

left_wheel_velocity: f32
right_wheel_velocity: f32

Unicycle

Command

Unicycle(UnicycleCommand)
Holonomic(HolonomicCommand)

StateEstimator

post_init(Node)
prediction_step(Node, Option<Command>, time: f32)
correction_step(Node, [Observation], time: f32)
world_state() -> WorldState
next_time_step() -> f32
pre_loop_hook(Node, time: f32)

WorldState

ego: Option<State>
objects: dict(String, State)
landmarks: dict(i32, State)
occupancy_grid: Option<OccupancyGrid>

State

pose: (f32, f32, f32)
velocity: (f32, f32, f32)

PythonEstimator PerfectEstimator ExternalEstimator

Observation

sensor_name: String
observer: String
time: f32
sensor_observation: SensorObservation

SensorManager

Sensor

post_init(Node, initial_time: f32)
get_observations(Node, time: f32) -> [Observation]
next_time_step() -> Option<f32>

GNSSSensor

GNSSObservation

pose: (f32, f32, f32)
velocity: (f32, f32)
applied_faults
frame

SpeedSensor

SpeedObservation

linear_velocity: f32
lateral_velocity: f32
angular_velocity: f32
applied_faults
frame

DisplacementSensor

DisplacementObservation

translation: (f32, f32)
rotation: f32
applied_faults
frame

OrientedLandmarkSensor

OrientedLandmarkObservation

id: i32
labels: [String]
pose: (f32, f32, f32)
height: f32
width: f32
applied_faults
frame

RobotSensor

OrientedRobotObservation

name: String
labels: [String]
pose: (f32, f32, f32)
applied_faults
frame

ScanSensor

ScanObservation

distances: [f32]
angles: [f32]
radial_velocities: [f32]
applied_faults
frame

ExternalSensor

ExternalObservation

observation: serde_json::Value

SensorObservation

SensorObservations

GNSS(GNSSObservation)
Speed(SpeedObservation)
Displacement(DisplacementObservation)
OrientedLandmark(OrientedLandmarkObservation)
OrientedRobot(OrientedRobotObservation)
Scan(ScanObservation)
External(ExternalObservation)

DisplacementSensorFilterType

Range(RangeFilter)
Python(PythonFilter)
External(ExternalFilter)

GNSSSensorFilterType

Range(RangeFilter)
Python(PythonFilter)
External(ExternalFilter)

OrientedLandmarkSensorFilterType

Range(RangeFilter)
Id(StringFilter)
Label(StringFilter)
Python(PythonFilter)
External(ExternalFilter)

RobotSensorFilterType

Range(RangeFilter)
Id(StringFilter)
Label(StringFilter)
Python(PythonFilter)
External(ExternalFilter)

ScanSensorFilterType

Range(RangeFilter)
Python(PythonFilter)
External(ExternalFilter)

SpeedSensorFilterType

Range(RangeFilter)
Python(PythonFilter)
External(ExternalFilter)

FaultModel

post_init(Node, initial_time: f32)
add_faults(time: f32, seed: f32, observations: mut [SensorObservations], environment: Environment)

Additive

SensorVariables

apparition: Bernouilli
distributions: "*" DeterministRandomVariable
proportional_to: SensorVariable
proportional_factor: f32

Clutter

SensorVariables

apparition: "*" DeterministRandomVariable
distributions: "*" DeterministRandomVariable
observation_id: String

Misassociation

apparition: Bernouilli
distribution: DeterministRandomVariable

Misdetection

apparition: Bernouilli

ExternalFault PythonFaultModel

SensorFilter

post_init(Node)
filter(time: f32, observations: mut SensorObservations) -> Option(SensorObservations)

RangeFilter

SensorVariables

inside: bool

min_range: list(f32)
max_range: list(f32)

ExternalFilter

PythonFilter StringFilter

Environment

map: Map
meta_data_list: dict(String, NodeMetaData)

get_observable_landmarks(position, observer_height, max_distance) -> [OrientedLandmark]
is_target_observable(target_position, target_height, observer_position, observer_height, max_distance) -> bool

Map

landmarks: [OrientedLandmark]

OrientedLandmark

id: i32
labels: [String]
height: f32
width: f32

pose: (f32, f32, f32)

Navigator

post_init(Node)
compute_error(Node, WorldState) -> ControllerError
pre_loop_hook(Node, time: f32)
next_time_step() -> Option<f32>

TrajectoryFollower GoTo ExternalNavigator PythonNavigator

ControllerError

lateral: f32
longitudinal: f32
theta: f32
velocity: f32

Controller

post_init(Node)
make_command(Node, ControllerError, time: f32) -> Command
pre_loop_hook(Node, time: f32)
next_time_step() -> Option<f32>

PythonController PID ExternalController

NetworkManager

broker

process_messages()

Network

range: f32
reception_delay: f32
broker: Broker

make_channel(PathKey) -> PathKey
subscribe_to(keys: [PathKey], multiclient: Option<MultiClient>) -> MultiClient
send_to(key: PathKey, message: Envelope, time: f32)

Envelope

from: String
message: MessageTypes
timestamp: f32
message_flags: [MessageFlag]

MessageTypes

String(String)
GoTo(GoToMessage)
SensorTrigger(SensorTriggerMessage)
Observation([Observation])
Event(EventRecord)
Custom([u8])

MessageFlag

God
Unsubscribe
Kill

MultiClient Broker

Scenario

Event

triggering_nodes: [Regex]
trigger: EventTriggerConfig
event_type: EventTypeConfig

EventTriggerConfig

Time(TimeEventTriggerConfig)
Proximity(ProximityEventTriggerConfig)
Area(AreaEventTriggerConfig)

EventTypeConfig

Spawn(SpawnEventConfig)
Kill(String)

TimeEventTriggerConfig

tim e
occurences

ProximityEventTriggerConfig

protected_target: Option<String>
distance: f32
inside: bool

AreaEventTriggerConfig

Rect(RectAreaEventTriggerConfig)
Circle(CircleAreaEventTriggerConfig)

RectAreaEventTriggerConfig

inside: bool

bottom_left: (f32, f32)
top_right: (f32, f32)

CircleAreaEventTriggerConfig

radius: f32
inside: bool

center: (f32, f32)

SpawnEventConfig

model_name: String
node_name: String

Drawable

add_record(time: f32, record: RecordType)
draw(ui, viewport, painter_info, scale: f32, time: f32) -> ([Shape], (f32, f32))
react(ui, ctx, response, painter_info, scale: f32, time: f32)

PluginAPI

get_state_estimator(custom_config) -> StateEstimator
get_navigator(custom_config) -> Navigator
get_controller(custom_config) -> Controller
get_physics(custom_config) -> Physics
get_sensor(custom_config) -> Sensor
get_sensor_filter(custom_config) -> SensorFilter
get_sensor_fault(custom_config) -> FaultModel
get_drawable(custom_config) -> Drawable

*

?

?

+

*

+

*

*

*

+

+

+

*

* *
*

*

* *

*

*